Supplementary Materials: Benchmarking Planning in AI Coding Agents: A Cross-Model Study of Four Planning Surfaces

Luis Herrera-Izquierdo [1,*]

This document contains the full rubric definitions, per-leg metadata, cost and wall-time tables, the plan-defect taxonomy, the procedural-deviation log, the cross-model robustness detail, and the canonical-pairing (`opencode`) analysis for the main article. It corresponds to Appendices A–G of the source study; the supporting artifacts are available at https://github.com/herrera-luis/agent-plan-comparison-supplementary. Table and figure numbers are prefixed "S"; references of the form "Table 1" or "Section 3" point to the main article.

1. Full rubric with anchors

This appendix reproduces the frozen sixteen-parameter rubric in full. It is the source-of-truth for every score in `scoring/scores.csv`; weights sum to 100. Parameters 1–10 are the engineering-rigor artefact-and-process dimensions; their relative weights are scaled so that the ten of them sum to 70. Parameters 11–16 are the six tool-surface dimensions (Section 2.2) that prevent the rubric from biasing toward tools that ship persistent multi-file artefacts; each carries a flat weight of 5 (the six sum to 30). Each parameter carries five integer anchors. When a run sits between two anchors the scorer prefers the lower.

1. Artifact completeness (weight 7)

Does the planner produce the *shape* of artefacts a downstream implementer needs (proposal, design, requirements, task list), or only a chat blob?

1 A single chat-resident blob; no separable artefacts.

2 A single markdown document, sectioned but not split.

3 Two artefacts (e.g. a plan doc plus a task list) with light cross-linking.

4 Three or more artefacts but missing one of {proposal, design, requirements, tasks}.

5 Multi-file structured set covering proposal, design, requirements/specs, and tasks.

2. Requirement traceability (weight 10.5)

Can a future reader trace each task back to a specific requirement?

1 Tasks have no link to requirements; "implement X" with no reference.

2 Free-text references to requirements but no IDs.

3 Some tasks cite requirement names but not all.

4 Every task cites a requirement, but the format is inconsistent.

5 Every task carries an explicit `Covers: <REQ-ID>` (or equivalent stable ID) and the IDs resolve to scenarios in the spec.

3. Task granularity (weight 7)

Are tasks sized for individual review and check-off?

1 One mega-task ("implement the feature") or no task list.

2 A handful of multi-day chunks; not individually reviewable.

3 $\sim$ 10 medium tasks; some are still vague.

4 $\sim$ 20 tasks; most are concrete but a few are still mega-tasks.

5 Tasks scoped to $\leq$ 30 minutes of work each; every task is a single PR-sized unit.

4. Test-plan coverage (weight 10.5)

Does each task carry a test plan a reviewer can execute?

1 "Add tests" appears once globally with no specifics.

2 A separate "tests" task at the end with no per-task linkage.

3 Some tasks include a sentence-level test note.

4 Most tasks include a concrete test plan, but coverage gaps remain.

5 Every task ships with a per-task test plan (commands or scenario refs); test plans cite real fixtures or scenarios.

5. Code-reference accuracy (weight 7)

Are file paths, function names, and APIs cited in the plan real, or hallucinated?

1 Referenced paths or APIs do not exist in the live repo.

2 Some references are real, but at least one is hallucinated.

3 All references are plausible; spot-check finds them mostly correct but with stale signatures.

4 All file paths verified; minor naming drift in one or two function references.

5 Every cited file/function/API is verifiable in the live repo at the planner's stated path.

6. MCP / external-source integration (weight 7)

When the prompt references external systems (Jira ticket, GitHub PR), does the planner actually read them?

1 Planner ignores the external link entirely.

2 Planner names the external system but quotes nothing from it.

3 Planner summarises the external source but with at least one factual error.

4 Planner integrates the external source faithfully; minor omission.

5 Planner reads, summarises, and cites the external system correctly; integrates ticket comments and linked code into the plan.

7. Reproducibility (artefact persistence), weight 7

If the user closes the chat, can the plan be re-opened verbatim?

1 Plan lives only in volatile chat history; no on-disk artefact.

2 User can manually copy the plan out, but no native export.

3 Plan has a native on-disk surface, but no version-control integration.

4 Plan is on disk and gits cleanly, but lacks a validation/lint step.

5 Plan is a versioned directory in git with a CLI-driven validator (`openspec validate --strict` or equivalent).

8. Latency to "ready to implement" (weight 3.5)

Wall-clock time from prompt-submitted to a complete plan being ready.

1 $>$ 60 minutes.

2 30–60 minutes.

3 15–30 minutes.

4 10–15 minutes.

5 $<$ 10 minutes.

9. Token / API cost (weight 3.5)

Approximate API spend taken from `metadata.json.token_estimate`.

1 $>$ \$5.00.

2 \$2.00–\$5.00.

3 \$1.00–\$2.00.

4 \$0.50–\$1.00.

5 $<$ \$0.50.

10. Hand-off friction to implementation (weight 7)

Once the plan exists, how much friction is there to start coding from it?

1 Manual re-typing required; no path to feed the plan back to a coding agent.

2 Plan can be copy-pasted into a fresh agent prompt, but no first-class hand-off tool.

3 One-step manual hand-off (copy to file, point an agent at it).

4 Native hand-off via a documented command, requiring one explicit user action.

5 Single-command hand-off (`/opsx-apply`, Cursor $\rightarrow$ Agent Mode, Ralph loop) with no manual re-typing.

11. Tool licensing & accessibility (weight 5)

Is the tool itself open-source, and are the features that this study exercises (MCP transports, agent mode, plan-card hand-off) reachable on a free tier or behind a paid plan? Anchor evidence is drawn from the public pricing pages [1,2].

1 Closed-source *and* the primary feature exercised by the study (chat MCP, agent mode) is gated behind a paid plan with no free fallback.

2 Closed-source freemium; the study's primary feature is gated behind a paid plan (e.g. Cursor's MCP picker requires Pro at \$20/mo).

3 Closed-source freemium where most features are reachable on the free tier and only premium model rates require paid plans.

4 Open-source frontend; some advanced features only available in a paid hosted offering or with subscription credits.

5 Fully open-source; all features used by this study reachable on the free tier; BYOK supported (Cline; OpenSpec and Ralph as free CLIs / loops with no licensing layer).

12. Learning curve / time-to-productivity (weight 5)

How much friction does the operator face between "never used the tool" and "producing a usable plan"? This dimension is distinct from O1 (setup friction) because it scores the cognitive cost of the *plan format and conventions* the operator must learn, not just the install pipeline.

1 Multi-step CLI install plus multiple file conventions plus a non-trivial mental model the operator must internalise (sprint budgets, validators, lifecycle states) before the first plan.

2 CLI install plus two coupled conventions to learn before any plan content (OpenSpec: install + `openspec init` + 4-artefact mental model + `validate -strict` semantics).

3 Single setup step plus one small mental model (Ralph: `RALPH_TASK.md` + loop iteration semantics).

4 Already-in-IDE chat surface plus one micro-convention to learn (e.g. a slash command or mode switch).

5 Zero install, zero conventions; chat is the primary surface and the tool drives the format end-to-end (Cursor in Plan Mode).

13. Human-in-the-loop validation experience (weight 5)

How easily can the operator inspect, refine, and correct the plan before it is handed to execution, without restarting the workflow? This parameter is *model-sensitive*: the underlying tool surface sets the ceiling, but the model's habit of surfacing clarifying questions versus committing to a single best guess shifts the score within that ceiling.

1 No mid-flight refinement; any plan change requires restarting the workflow from scratch with a new prompt.

2 Plan can be edited but the tool must be re-invoked over the edited file (a headless CLI re-run over an edited plan file: edit-then-replay).

3 Plan can be edited in place; the tool re-reads on next invocation but offers no clarification affordance.

4 Plan-card hand-off plus in-thread refinement; the tool absorbs follow-up turns, but does not surface clarifying questions to the operator (Cursor's chat surface; OpenSpec/Ralph under Cline-as-extension, both under either model).

5 Plan-card hand-off plus in-thread refinement *and* the tool actively surfaces clarifying questions when the prompt is ambiguous, raising the operator's awareness of the choices being made (no cell in this study reaches anchor-5 on Param 13).

14. Autonomous capacity (weight 5)

Once the plan exists, how much wall-clock work can the tool do without the operator present, and how gracefully does it handle running out of budget or hitting an unrecoverable failure?

1 No headless mode; an operator must be present to advance every action and to dispatch every tool call (Cursor: even "Agent Mode" execution requires an attended chat panel).

2 Headless mode exists, but with no budget intelligence and no per-iteration commit (OpenSpec: end-to-end run drives the validator to convergence but does not chunk progress to git).

3 Headless mode plus per-iteration commit (each loop iteration lands as a commit), but with no budget intelligence.

4 Headless mode plus per-iter commit plus soft budget warnings when wall-clock or token spend approaches a configured ceiling.

5 Headless mode plus budget intelligence (cost ceilings, sprint-style budgets), per-iter commit, and an explicit `TASK_GUTTER` fallback when the loop cannot make further progress (Ralph: budgets, per-iter commit, and gutter fallback are all built in).

15. Runtime integration & ergonomics (weight 5)

How well does the tool fit into the operator's existing IDE / shell runtime, and how cleanly does it surface MCP transports and model selection without requiring a separate process?

1 CLI scaffold install plus file-only conventions; no chat surface and no integrated MCP runtime.

2 CLI install with a separate runtime; chat surface available only when paired with another tool (OpenSpec: paired with Cline-as-extension or with `cursor-agent` for execution).

3 Loop harness with optional in-IDE driver (Ralph: loop runs on its own; Cline-as-extension and `cursor-agent` both work as drivers).

4 In-IDE chat panel with integrated MCP transports; runs alongside the editor as a first-class extension.

5 Native to the IDE the operator already uses; integrated MCP picker; chat is the primary surface; model picker is one click; slash-commands are first-class (Cursor).

16. Code-review / hand-off review surface (weight 5)

Once a plan exists, how well can a non-author teammate (or a future operator) review it asynchronously?

1 Chat thread only; no on-disk artefact a non-author can fetch without operator help.

2 Chat thread plus a non-deterministic local save location (Cursor: `~/.cursor/plans/<hash>-pla` exists locally but the path is operator-specific).

3 Single file committed to git with no validator and no required structure.

4 Single file committed to git plus a non-strict structural convention reviewers can read mechanically (Ralph: `PLAN.md` sections + `TASK_COMPLETE` sentinel).

5 Multiple structured files committed to git plus a strict CLI validator (`openspec validate -strict`) so reviewers can audit completeness mechanically as well as visually.

Frozen-as-of. The rubric is frozen at the commit that introduces `rubric.md`. Parameters 11–16 were added in the revision that broadened the rubric to score tool-surface dimensions (licensing, learning curve, human-in-the-loop validation, autonomous capacity, runtime integration, and code-review surface). At introduction, every prior row was re-scored under the new weights to preserve the frozen-as-of contract: any change to parameters, weights, or anchors after a single row has been written to `scores.csv` requires re-scoring all prior rows.

2. Operator-experience companion rubric

The primary rubric in Appendix 1 is intentionally weighted toward dimensions that matter for sustained, hand-off-ready engineering work: artefact persistence, requirement traceability, and a strict validator. Lightweight surfaces such as a chat-resident plan look bad on those dimensions even when they shine on the operator's lived experience: setup time, cognitive load, time-to-first-result, and how easy it is to course-correct without restarting. To prevent the headline ranking from being read as a property of the rubric instead of a property of the tools, this appendix introduces a four-parameter *operator-experience* companion rubric with equal weights (25 % each, sum 100 %).

The companion rubric is scored *per tool*, not per (tool × benchmark): O1–O4 are properties of the tool surface, not of the task. Each parameter is scored on the same 1–5 anchor scale and defended in `scoring/grill-me-log.md`.

O1. Setup friction (weight 25)

How much work does the operator do before any plan content can appear?

1 Multi-step CLI install plus scaffold conventions and file layout the operator must learn before producing a usable plan.

2 CLI install plus a small number of one-time configuration files; conventions documented but still must be learned.

3 CLI install only; conventions are minimal or self-documenting.

4 No install; one configuration file (e.g. `.mcp.json`) is the entire setup.

5 Zero install: already in the IDE / shell the operator uses for other work.

O2. Operator cognitive load (weight 25)

How many sub-formats and conventions must the operator hold in head to author and review a plan?

1 Multiple coupled sub-formats (proposal vs design vs spec deltas vs tasks vs validator output); operator must remember which content belongs in which file.

2 Two-to-three coupled sub-formats with documented but non-trivial separation.

3 Single primary file plus one auxiliary surface (e.g. `PLAN.md` plus a task-completion signal).

4 Single primary surface; auxiliaries are lightweight and optional.

5 Single chat surface; the tool drives the format and the operator never has to recall a layout.

O3. Time-to-first-viable-plan (weight 25)

Wall-clock time from prompt-submitted to the first piece of usable plan content appearing on the operator's screen (not full plan completion, *first* content).

1 Several minutes of scaffolding (file creation, validator boot, MCP handshake) before any plan prose appears.

2 Roughly a minute of scaffolding before plan prose starts streaming.

3 Some scaffolding overhead but the operator can see the plan forming inside the first 30 seconds.

4 Plan content streams within seconds; minimal scaffolding.

5 Plan content streams immediately; the operator sees the model thinking on screen.

O4. In-flight iterability (weight 25)

How easy is it to refine the plan once it exists, without restarting the workflow?

1 Plan revisions require restarting the workflow from scratch.

2 Plan revisions require editing files by hand and re-running the validator/loop.

3 Plan revisions are supported via re-invocation against the same files.

4 Plan revisions are supported in-thread but require an explicit "revise" command.

5 Operator can refine the plan in the same chat without restarting; the tool absorbs follow-up turns.

Per-tool scores

The companion rubric is scored per tool (the four dimensions are properties of the tool surface, not of the task). Anchors are written defenses in `scoring/grill-me-log.md` (operator-experience section). Table S1 reports the per-parameter scores and the equal-weighted per-tool totals.

Table S1. Operator-experience scores (1–5 per parameter; weighted total at equal 25 % weights, max 5.0). Column abbreviations: O1 setup friction, O2 cognitive load, O3 time-to-first-viable-plan, O4 in-flight iterability. OpenSpec and Ralph are scored under Cline-as-extension, the driver used under *both* models, so these scores are model-stable. The OpenCode row is an anchor sketch pending a formal grill-me rescoring pass; its native headless operating mode constrains O4 to anchor-2 (revising a plan requires re-invocation and overwriting the session), which caps its OPX total well below Cursor's despite OpenCode leading the engineering-rigor rubric.

Tool	O1	O2	O3	O4	Total
`cursor`	5	4	5	5	**4.75**
`opencode`	3	3	4	2	**3.00**
`openspec`	2	2	3	3	**2.50**
`ralph`	2	2	3	2	**2.25**

The ranking inverts the engineering-rigor headline: Cursor wins the operator-experience rubric on the strength of zero-install setup (O1: 5), single chat surface (O2: 4), immediate plan streaming (O3: 5), and in-thread revision (O4: 5). OpenCode places second: a single-binary install plus a per-bench `opencode.json` is a light ramp (O1: 3), `opencode run` streams content immediately (O3: 4), and the operator holds only one plan file plus the `session-export.json` schema in mind (O2: 3); its only real weakness is that the headless-only operating mode admits revision only by re-invocation and session overwrite (O4: 2). OpenSpec places third: its 4-artefact scaffold imposes the highest file-format setup and cognitive-load costs (O1: 2, O2: 2), though driven via Cline-as-extension the agent thinking streams live in the chat panel (O3: 3) and revisions are supported by re-invoking against the same files (O4: 3). Ralph lands last on operator experience despite its lightweight single `PLAN.md`: operating it requires authoring `RALPH_TASK.md` (success criteria plus a verification command) and internalising the loop, budget, and `GUTTER`/`COMPLETE` semantics, the largest conceptual ramp of the four (O1: 2, O2: 2); the loop boots before the first plan prose appears (O3: 3), and refining the plan means editing the task file and re-running the loop rather than taking a follow-up turn in chat (O4: 2).

3. Per-leg metadata

Per-leg metadata is recorded in `metadata.json` files alongside each canonical artefact, organised one directory per (model, benchmark, tool, leg) cell under `runs/`. The fields relevant to the rubric are summarised in Table S2 (planning legs) and Table S3 (execution legs); both tables report all 48 (tool × benchmark × leg × model) primary cells under Claude Opus 4.8 and GPT-5.5.

Table S4 is the compact planning-leg view of the same data referenced from the main text (Section 3): wall time and verified live MCP-call counts per (tool, benchmark) cell under both models. Wall time is the `started_at` → `ended_at` clock span, so for the attended cells it includes operator idle time; the Opus 4.8 `openspec` video-support (23663 s) and single-file (2335 s) rows are attended-idle artefacts scored at anchor-5 on Parameter 8 rather than penalised (see Section 4.3).

Table S5 gives the per-cell token and dollar breakdown referenced from the cost analysis in the main text (Section 3): k-token volume and dollar cost for every (tool, bench-

Table S2. Planning-leg metadata under both primary models. Wall seconds and MCP-call counts come from per-leg `metadata.json` (`mcp_calls_summary`) when the driver emits a structured session export (the OpenCode rows under both models), and from per-call enumerations in `mcp-calls.md` (clean scoring view, backed by the operator-curated raw dump in `mcp-calls.txt`) for the operator-attended Cursor rows under both models. The Cursor cells that the operator did not attend (the single-file bench, plus all four execution legs) emit no MCP traffic by design and are reported "0". The OpenSpec and Ralph cells are driven through Cline-as-extension under *both* models and emit chat transcripts but no machine-parseable MCP envelope, so MCP counts on those rows are recorded "n/c" (not captured) under both columns; each tool used the same MCP servers as the corresponding Cursor / OpenCode cell (Atlassian + GitHub on the worker-video-feature video-support bench, GitHub-only on the github-repo-bootstrap single-file bench and the personal-blog-scaffold blog bench).

		Opus 4.8		GPT-5.5	
Benchmark	**Tool**	**W (s)**	**MCP**	**W (s)**	**MCP**
`github-repo-bootstrap`	`cursor`	76	0 (sibling)	83	0 (sibling)
	`openspec`	2335	n/c	181	n/c
	`ralph`	215	n/c	270	n/c
	`opencode`	98	1 (all GH)	119	5 (all GH)
`worker-video-feature`	`cursor`	119	12 (2 Atl + 10 GH)	119	16 (2 Atl + 14 GH)
	`openspec`	23663	n/c	269	n/c
	`ralph`	450	n/c	176	n/c
	`opencode`	293	17 (2 Atl + 15 GH)	188	16 (1 Atl + 15 GH)
`personal-blog-scaffold`	`cursor`	181	4 (all GH)	66	6 (all GH)
	`openspec`	1166	n/c	212	n/c
	`ralph`	507	n/c	253	n/c
	`opencode`	118	3 (all GH)	123	3 (all GH)

mark, leg) cell under both models, with each cell tagged by source ([r] rate-card vs [p] provider/cache-aware).

The complete `metadata.json` files (with full notes, token-metric caveats, MCP-call enumerations, and exit-reason explanations) are part of the released artefact set under `research/agent-plan-comparison-final/runs/`, organised as `runs/<model>/<benchmark>/<tool>`

Table S3. Execution-leg metadata under both primary models. Source: per-leg `metadata.json` plus the on-disk diff-stat for each cell. The Cursor single-file bench leg's validator runs a per-task verification harness with 7 of 10 executable checks passing under both models; 3 skip for environment reasons (no parent Terraform root module, no GitHub provider, no live target repository checkout). The GPT-5.5 Cursor video-support bench wall time of 325 s and the Opus 4.8 wall time of 599 s both surface the propose/apply two-phase workflow in the chat trace. The worker-video-feature GPT-5.5 OpenCode row at 9 files / +932 / −7 is dominated by the implementation diff (parser code + tests + spec / tasks.md updates); the 12 files / +418 / −5 Cursor video-support bench Opus 4.8 row similarly includes the OpenSpec change-directory files plus an init-generated `openspec/config.yaml` materialised by the propose phase. The github-repo-bootstrap single-file bench rows at 15–19 files / +382–574 / −59 on the GPT-5.5 OpenSpec / Ralph / OpenCode tools reflect a repo-wide `terraform fmt -recursive` the GPT-5.5 planner ran to clear pre-existing fmt drift in 13 unrelated files (Appendix 4.4); the corresponding Opus 4.8 rows on those tools deliberately scoped that fix out and are 2–7 files with no deletions. The `personal-blog-scaffold` rows are greenfield builds from an empty repository, so they are near-pure insertion (29–39 files, ∼1,000–1,700 new lines, ≤ 2 deletions) and every leg's validator is `astro build`; the GPT-5.5 OpenSpec blog row at +9074 committed the generated lockfile alongside source.

		Opus 4.8				GPT-5.5				
Bench	**Tool**	**W**	**F**	+	−	**W**	**F**	+	−	**Validator**
single-file	`cursor`	76	1	254	0	210	1	254	0	7/10 PASS, 3 SKIP
single-file	`openspec`	499	7	534	0	508	19	574	59	openspec exit=0
single-file	`ralph`	221	2	363	0	175	15	492	59	TASK_COMPLETE
single-file	`opencode`	235	2	434	0	184	15	382	59	terraform fmt clean
video-support	`cursor`	599	12	418	5	325	11	447	3	vitest green
video-support	`openspec`	439	19	1823	3	328	10	830	7	vitest green
video-support	`ralph`	514	8	584	6	210	7	446	3	vitest green
video-support	`opencode`	350	6	624	4	253	9	932	7	vitest green
blog	`cursor`	1179	31	1410	2	610	23	1160	2	astro build OK
blog	`openspec`	1517	36	1561	1	576	34	9074	1	astro build OK
blog	`ralph`	1744	39	1678	2	1045	24	981	2	astro build OK
blog	`opencode`	1724	29	1649	2	1245	34	1259	2	astro build OK

Table S4. Planning-leg wall time (s) and verified live MCP calls under both models. `openspec` and `ralph` run operator-attended via Cline-as-extension, which emits no machine-parseable MCP envelope, so their MCP counts are "n/c" (not captured). Cursor MCP rows are operator-curated from the chat trace (clean view `mcp-calls.md`, raw dump `mcp-calls.txt`); OpenCode rows come from its session export. "Pinned" marks legs whose MCP calls carry the pre-feature reference-state pin.

		Opus 4.8			GPT-5.5		
Tool	Benchmark	Wall (s)	MCP	Pinned	Wall (s)	MCP	Pinned
`cursor`	single-file	76	0	—	83	0	—
`openspec`	single-file	2335	n/c	yes	181	n/c	yes
`ralph`	single-file	215	n/c	yes	270	n/c	yes
`opencode`	single-file	98	1	yes	119	5	yes
`cursor`	video-support	119	12	yes	119	16	yes
`openspec`	video-support	23663	n/c	yes	269	n/c	yes
`ralph`	video-support	450	n/c	yes	176	n/c	yes
`opencode`	video-support	293	17	yes	188	16	yes
`cursor`	blog	181	4	yes	66	6	yes
`openspec`	blog	1166	n/c	yes	212	n/c	yes
`ralph`	blog	507	n/c	yes	253	n/c	yes
`opencode`	blog	118	3	yes	123	3	yes

Table S5. Per-cell token estimates (k-tokens) and dollar cost under both models. `[r]` = rate-card estimate (token volume × weighted rate: Opus 4.8 $19/M, GPT-5.5 $22.5/M); `[p]` = provider/cache-aware (`metadata.json cost_estimate_usd`). Source: `tools/compute_costs.py` and `tools/dump_dual_primary.py`.

		cursor [r]				openspec [r]				ralph [r]				opencode [p]			
		Opus		G5.5		Opus		G5.5		Opus		G5.5		Opus		G5.5	
B	L	kt	$	kt	$	kt	$	kt	$	kt	$	kt	$	kt	$	kt	$
s	p	67.6	1.28	43.2	0.97	76.6	1.46	38.4	0.86	78.0	1.48	53.3	1.20	1199.0	4.59	784.3	1.00
s	e	75.1	1.43	60.2	1.35	123.5	2.35	107.8	2.43	102.8	1.95	70.1	1.58	4598.7	5.13	1436.5	1.45
c	p	105.1	2.00	74.5	1.68	92.3	1.75	70.0	1.57	94.0	1.79	55.2	1.24	6870.3	12.07	1396.8	1.70
c	e	165.7	3.15	129.7	2.92	170.9	3.25	119.2	2.68	203.9	3.87	96.7	2.18	8231.9	10.00	2193.0	2.15
b	p	73.2	1.39	35.5	0.80	50.3	0.96	30.0	0.67	53.7	1.02	23.1	0.52	694.5	2.98	437.9	1.09
b	e	209.4	3.98	79.7	1.79	173.4	3.30	72.0	1.62	196.0	3.72	90.3	2.03	39454.5	46.59	6519.5	7.29
s tot		142.7	2.71	103.4	2.33	200.1	3.81	146.2	3.29	180.8	3.43	123.4	2.78	5797.7	9.72	2220.8	2.45
c tot		270.8	5.15	204.2	4.59	263.2	5.00	189.2	4.26	297.9	5.66	151.9	3.42	15102.2	22.07	3589.8	3.85
b tot		282.6	5.37	115.2	2.59	223.7	4.26	102.0	2.29	249.7	4.74	113.4	2.55	40149.0	49.57	6957.4	8.38
3-bench		696.1	13.23	422.8	9.51	687.0	13.05	437.4	9.84	728.4	13.84	388.7	8.75	61048.9	81.36	12768.0	14.68

4. Procedural deviations

Two procedural deviations from the prescribed protocol were observed on the operator-attended Cursor video-support bench execution legs during the study. Both are model-independent: one is a property of the operator's local shell environment, the other of the two-phase Cursor OpenSpec workflow, and neither recurs in the frozen Opus 4.8 Cursor capture (whose execution leg records zero plan deviations). They do not move the rubric scores; they are recorded as evidence about the operational fragility of an attended workflow vs a headless one.

C.1 Shell-environment quirk on the video-support bench Cursor execution leg

A zsh `chpwd` hook (visible as `_encode:25: command not found` in shell stderr) prevented `cd` from working inside sandboxed shell sessions. Mid-execution this caused several `git` commands to silently target the wrong repo (the workspace repo rather than the target repo). Recovered by switching to `git -C <abs-path>` for all repo operations and verifying branch placement before continuing. No source content was lost (file edits used absolute paths). The quirk is a property of the operator's local zsh environment, not of any one chat session.

C.2 Workspace-routing quirk on two-phase `opsx-propose` / `opsx-apply` executions

On the captured run, the propose phase was first opened in the `agent-research` workspace (the workspace where the planning chat had been authored), so the agent initially scaffolded the OpenSpec change directory under that workspace's `openspec/changes/` tree, not under the target `core-services` checkout where the worker code lives. The operator detected the mis-route from `git status`, re-ran `openspec init <core-services-path> -tools cursor -force`, moved the change directory across, and started the apply phase in a fresh Cursor window rooted at the target repo. The workflow trap is specific to two-phase OpenSpec executions in Cursor: the first phase implicitly inherits the workspace of the chat that owns the plan card, and the operator must explicitly re-root before phase two or the artefacts land in the wrong tree. It is a property of the two-phase Cursor workflow rather than of any one model.

C.4 `terraform fmt -recursive` scope split on the single-file bench cells (cross-model)

The single-file bench prompt asks the planner to add one Terraform module declaration and verify the post-write tree formats cleanly via `terraform fmt -check -recursive`. The target repo carries 13 pre-existing unformatted files unrelated to the requested change (unrelated service directories under `aws-identity-center/`, `aws-tag-policies/`, `poc/SRE-6603/`, etc.). The canonical-pairing split surfaces on three of the four GPT-5.5 single-file bench cells (OpenSpec, Ralph, OpenCode) and on neither of their Opus equivalents.

- *Opus single-file bench / OpenSpec, Ralph, OpenCode* kept the change scope tight. The planner recognised that the 13 unrelated files were owned by separate service directories per `AGENTS.md`, ran `terraform fmt -check -recursive github/repositories/sre/` (exit 0) and `terraform fmt -check -diff` on the new file itself (exit 0), and recorded the global drift as a pre-existing condition in `deviations.json` without auto-reformatting. The execution-leg diffs are 6 files / +521 / −0 (OpenSpec), 2 files / +442 / −0 (Ralph), and 2 files / +394 / −0 (OpenCode).
- *GPT-5.5 single-file bench / OpenSpec, Ralph, OpenCode* all ran a repo-wide `terraform fmt -recursive` from the repo root to clear all 13 files of drift before re-running `terraform fmt -check -recursive` for the strict-clean postcondition. The execution-leg diffs are therefore 15 files / +574 / −59 (OpenSpec), 15 files / +492 / −59 (Ralph),

and 15 files / +382 / −59 (OpenCode), with the 13 unrelated files contributing the bulk of the deletions on every row. The recorded `deviations.json` entries frame this as a precondition mismatch that the planner resolved unilaterally.

- *Cursor / both models* kept the change scope tight on both cells (1 file / +254 / −0 under both Opus and GPT-5.5), because the single-file bench Cursor cells produce a Terraform module declaration in a sibling repo whose own root-module Terraform tree is clean.

Both response patterns produce a green strict-clean postcondition; neither violates the test plan. The cross-model split is consistent across canonical-pairing tools (OpenSpec, Ralph, OpenCode all behave the same way under each model), confirming the asymmetry is model-bound rather than tool-bound: GPT-5.5 pulls scope outward toward strict-postcondition closure, Opus pulls scope inward toward single-file-change minimality. The execution-leg metadata table (Table S3) carries the diff-size asymmetry explicitly.

C.5 Why these deviations matter

The deviations contribute to the broader cross-tool defect-convergence story discussed in Section 4. Across the eight (tool × leg) cells the study surfaces *five* distinct defect categories: a regex passthrough trap (cursor and ralph under both models, OpenCode under GPT-5.5 only; Appendix 5), a plan-miscount on the single-file bench (OpenCode under Opus 4.8; Appendix 5), a shell-environment quirk (cursor, this appendix), a workspace-routing quirk specific to two-phase Cursor executions (cursor, this appendix), and a cross-model scope split on a precondition-mismatch fix (OpenSpec, Ralph, OpenCode under GPT-5.5; this appendix). The distribution of defects across surfaces is itself a finding: the operator-attended workflow accumulates the most distinct defect classes; the headless and Cline-driven workflows accumulate fewer distinct classes but heavier-tail individual defects (the regex passthrough trap reaches OpenSpec not at all, is caught by Ralph and OpenCode/GPT-5.5 through Node REPL pre-checks or vitest failures, and is avoided up front by OpenCode under Opus 4.8); the cross-model scope-resolution split appears on every canonical-pairing single-file bench cell under GPT-5.5 but on none of their Opus 4.8 equivalents, confirming that asymmetry is model-bound rather than tool-bound.

5. Plan-defect taxonomy

We catalogue plan defects in three categories ordered from most to least diagnostic of plan quality. Counts are per (tool × leg × model) cell on the video-support benchmark unless otherwise noted.

Category 1: Plan-was-wrong.

The plan asserts a falsifiable factual claim about the target system that empirically does not hold.

Found in 5 of 8 (tool, model) planning legs on the video-support bench (Table 2 in Section 3.2). The Cursor planning legs under both models and the Ralph planning legs under both models all assert that `STANDARD_PATH_REGEX` "already accepts any `[a-z]+` extension" (Cursor's plan Section 2: "No structural change needed: `STANDARD_PATH_REGEX` already captures `[a-z]+` extensions"; Ralph's `PLAN.md` claims the `\.([a-z]+)` capture already matches mp4/webm/mov). The two OpenCode planning legs diverge by model: under **Opus 4.8** the plan body explicitly diagnoses the defect: "`mp4` contains a digit and fails the regex … the group must become `([a-z0-9]+)`" (`OPENCODE_PLAN.md` Section 2.3), with Task 2 making the one-character broadening before any code is written, and therefore *avoids* the trap; under **GPT-5.5** the plan body instead claims `[a-z]+` "already allows

mp4/webm/mov" and falls in. The digit `4` in `mp4` falls outside the character class; the regex returns `null` at runtime. The Cursor execution legs discover the mismatch when running the test suite (the parser is broadened during apply); the Ralph execution legs discover it via an empirical pre-implementation Node REPL check (Opus 4.8) or a Vitest failure (GPT); the OpenCode/GPT execution leg catches it via a Vitest failure and broadens the character class (`[a-z]+` → `[a-z0-9]+`). OpenSpec's `design.md`/spec avoids the assertion entirely under both models.

Reading. The trap lands in 5 of 8 (tool, model) video-support bench cells: on Cursor and Ralph under both models, never on OpenSpec, and on OpenCode under GPT-5.5 only. The Opus 4.8 captures expose *two distinct avoidance mechanisms* (Section 3.2): OpenSpec's 4-artefact spec format, which front-loads parser broadening as a first-class requirement and avoids the trap under both models without inspecting the digit; and OpenCode/Opus 4.8's plan precision, which diagnoses the digit directly and is the single most precise code-reference grounding in the study, but is model-dependent: the same tool falls into the trap under GPT-5.5. High plan-content quality can therefore avoid the defect when it is high enough, but only the structural spec format avoids it under *both* models.

A distinct category-1 instance specific to the OpenCode/Opus 4.8 video-support cell concerns runtime behaviour rather than a regex: the plan proposes detecting an unsatisfiable Range from `object.range` after `BUCKET.get(key, {range})`. The execution leg empirically probed the Miniflare/workerd R2 runtime and found that an out-of-bounds Range is *silently clamped* (the get returns the full object with `range {offset:0, length:size}`), so the planned out-of-bounds check can never fire; the fix parses the client Range header directly against `object.size` and returns 416 with `Content-Range: bytes */{size}`. The same R2-clamping discovery is recorded in the Ralph/Opus 4.8 video-support execution leg. Both are noted in the respective `deviations.json`.

A *lighter* category-1 instance appears on the OpenCode single-file bench under Opus 4.8: the planning leg's plan body claims `team_access` has 47 entries where the frozen prompt body's inlined sibling actually has 53. The OpenCode execution leg honoured the verbatim-copy requirement (producing all 53 entries) and recorded the count error in `deviations.json`, dropping the cell's Param 5 Code-reference accuracy to 4. Cursor states the correct 53 under Opus 4.8. Neither OpenSpec nor Ralph emits a count error under either model; the defect is planner-local rather than benchmark-wide.

Category 1b: Greenfield completeness omissions.

The blog bench has no live code to misread, so its category-1-adjacent defect is *omission*: a part of the bilingual contract the plan never specifies. The omissions cluster under GPT-5.5 and are absent under Opus 4.8. *(i) Missing `x-default`.* The GPT-5.5 OpenSpec spec pins only `hreflang` `en`/`es` and omits the `x-default` alternate that the Opus 4.8 OpenSpec spec names explicitly. *(ii) Missing article-index route.* The GPT-5.5 Cursor plan carries an "Articles" nav entry but never declares the `[locale]/articles/index` route or a file path for the root-locale redirect, both present in the Opus 4.8 Cursor plan. *(iii) Capability-matrix collapse.* The GPT-5.5 Ralph plan replaces the Opus 4.8 C1–C26 capability matrix (with per-task `Caps:` back-links and per-test capability citations) with a free-text bullet list, dropping Param 2 (Requirement traceability) from 4 to 2 and Param 4 (Test-plan coverage) from 5 to 4, the largest single cross-model swing in the study (-0.350). *Reading.* This is the mirror image of the regex trap: there a structural spec format is the model-robust avoidance mechanism, whereas here the stronger *model* (Opus 4.8) is the robust completeness driver and even the structural format (GPT-5.5 OpenSpec) drops `x-default`. Both

families reduce to whether the planner promotes an easily-forgotten sub-requirement to a first-class artefact.

Category 2: Plan-flagged-extraction.

The plan pre-flags a contingency that may need to be extracted into a separate task; the executor confirms and extracts. The 416 Range-Not-Satisfiable response path is the clearest example. OpenSpec includes the 416 path as a first-class scenario in its spec under both models; the OpenCode/Opus 4.8 plan proposes a dedicated `createVideoResponse` path for it; the Cursor video-support planning leg surfaces it as a discrete concern in the plan body; the Ralph plan does not pre-flag it and the executor fills the gap at apply time.

Category 3: Plan-underspecified.

The plan does not specify a runtime-specific behaviour; the executor fills the gap during testing. *Found in 2 of 6 (tool, model) execution legs*: the OpenSpec video-support bench execution legs under both Opus and GPT add strict response-header assertions (e.g. `Cache-Control`, `Vary`) that the spec did not pin; the test file's strict-equality assertions materialise the underspecified contract under both models.

Discussion.

The defect distribution across the three plan shapes is informative and largely *tool-bound*: chat-resident plans (Cursor) accumulate the most distinct defect classes (categories 1 and 2 plus the procedural deviations of Appendix 4); the 4-artefact spec (OpenSpec) accumulates the fewest planning-time defects but accumulates underspecification at execution time under both models; `PLAN.md`-shaped (Ralph) accumulates the same regex passthrough as Cursor on every video-support bench cell. The picture is largely robust to the model substitution: the trap-vs-avoid split is determined primarily by the spec format, with one model-dependent exception: OpenCode, whose Opus 4.8 plan is precise enough to avoid the trap that the same tool falls into under GPT-5.5. Whether the trap-avoidance signal generalises to other code-reference shallow defect classes is future work (Section 5).

6. Single-file bench results

This appendix reports the per-parameter score matrix (Table S6) for `github-repository-boot` under both model conditions, the benchmark whose prompt inlines the sibling Terraform module verbatim and therefore does not exercise external MCP integration.

The single-file bench has no MCP discriminator under either model because the prompt inlines the sibling `agent-skillzz.tf` HCL module directly, removing the need for any external read. The four GitHub `get_file_contents` calls that OpenSpec and Ralph made and the single `get_file_contents` call that OpenCode made are recorded as SHA-pin verification of the sibling content, not as substantive plan input; Cursor made no MCP calls at all under either model. Parameter 6 is therefore a 3 across all eight (tool, model) cells on this benchmark and the row carries no diagnostic weight here.

The discriminating parameters on the single-file bench split into two groups. In the artefact-and-process group (1–10), `openspec` saturates the structural anchors: Parameters 1 (Artifact completeness: 2/5/2/2), 4 (Test-plan coverage: 3/5/4/5), and 7 (Reproducibility: 3/5/4/4), because the 4-artefact change directory plus strict validator force structural content the chat-resident and single-file plans never produce. OpenCode picks up the plan-content anchors: Param 3 (Task granularity: 3/4/4/5), Param 4 (Test-plan coverage: tied with OpenSpec at 5), and Param 5 (Code-reference accuracy: 4/3/3/4 under Opus 4.8, with OpenCode's 4 reflecting the count error), which partly offset the structural deficit on the weighted total. The tool-surface group (11–16) splits the rubric across all four tools: `cursor` owns Parameters 12 (Learning curve) and 15 (Runtime integration);

Table S6. Per-parameter score matrix on the sixteen-parameter rubric for `github-repository-bootstrap` under both models. Rows 1–10 are the artefact-and-process parameters (weights scaled to sum to 70); rows 11–16 are the tool-surface parameters (each weight 5). The MCP row is a uniform 3 across all eight (tool, model) cells because the prompt inlines the sibling. The rows that shift between models are Parameter 5 (Code-reference accuracy: OpenCode lifts 4→5 under GPT-5.5 because the Opus 4.8 plan carries the 47-vs-53 `team_access` count error, while Cursor drops 4→3) and Parameter 9 (Token cost: Cursor and OpenSpec each lift 3→4 under GPT-5.5, OpenCode lifts 2→4, Ralph is flat at 3/3). Parameter 8 (Latency) is 5/5 for OpenSpec: although its Opus 4.8 planning leg records a 2335 s raw wall span, that span is an attended-session measurement artefact (Section 4.3) scored anchor-5, matching its 181 s GPT-5.5 leg. Parameter 13 (Human-in-the-loop validation) does *not* shift on this benchmark because OpenSpec and Ralph run Cline-as-extension under *both* models (the driver is held constant). Source: `scoring/scores-opus48.csv` and `scoring/scores-gpt55.csv`.

		cursor		openspec		ralph		opencode	
Parameter	**Wt**	Opus	G-5.5	Opus	G-5.5	Opus	G-5.5	Opus	G-5.5
1. Artifact completeness	7	2	2	**5**	**5**	2	2	2	2
2. Requirement traceability	10.5	1	1	2	2	1	1	2	2
3. Task granularity	7	3	3	4	4	4	4	**5**	**5**
4. Test-plan coverage	10.5	3	3	**5**	**5**	4	4	**5**	**5**
5. Code-reference accuracy	7	4	3	3	3	3	3	4	**5**
6. MCP integration	7	3	3	3	3	3	3	3	3
7. Reproducibility	7	3	3	**5**	**5**	4	4	4	4
8. Latency	3.5	**5**	**5**	**5**	**5**	**5**	**5**	**5**	**5**
9. Token cost	3.5	3	4	3	4	3	3	2	4
10. Hand-off friction	7	**5**	**5**	**5**	**5**	**5**	**5**	**5**	**5**
11. Tool licensing & accessibility	5	2	2	**5**	**5**	**5**	**5**	**5**	**5**
12. Learning curve / time-to-productivity	5	**5**	**5**	2	2	3	3	3	3
13. Human-in-the-loop validation	5	4	4	4	4	4	4	2	2
14. Autonomous capacity	5	1	1	2	2	**5**	**5**	2	2
15. Runtime integration & ergonomics	5	**5**	**5**	2	2	3	3	3	3
16. Code-review / hand-off review surface	5	2	2	**5**	**5**	4	4	4	4
Weighted total	100	**3.050**	**3.010**	**3.765**	**3.795**	**3.475**	**3.465**	**3.540**	**3.675**

`openspec` owns Parameter 16 (Code-review surface) at 5 alone, with Parameter 11 (Licensing) tied at 5 across OpenSpec, Ralph, and OpenCode; `ralph` owns Parameter 14 (Autonomous capacity). Parameter 13 (Human-in-the-loop validation) is model-stable on this benchmark: OpenSpec, Ralph, and Cursor all sit at 4/4 (the Cline-as-extension and chat surfaces expose the in-flight plan without a clarifying-question affordance under both models), and OpenCode stays at 2/2 under its native headless runtime. The end-to-end gap between `openspec` and `cursor` on this benchmark under Opus 4.8 is 0.715 on the full sixteen-parameter rubric; under GPT-5.5 the gap is 0.785. OpenCode lands below OpenSpec under both models (3.540 vs 3.765 under Opus 4.8, 3.675 vs 3.795 under GPT-5.5), so OpenSpec holds the single-file bench lead under either model.

7. Cross-model details

This appendix collects the per-parameter, per-cell details behind the unified two-model results in Section 3. The headline tables in the main text already report the per-cell weighted totals (Table 1), the unified cost picture (Table S5), the wall-time and MCP volume (Table S4), and the operator-experience companion rubric (Table 3). What follows is the supporting detail: per-million-token rates, the per-parameter Δ matrix, and the operator-experience driver substitution explained.

7.1. Per-million-token rates

Table S7 lists the per-million-token rates and the 30/70 input/output mix used to convert token estimates to dollars.

Table S7. Per-million-token rates for the two model conditions used in this study. The 30/70 mix is the input/output split this paper assumes when computing per-leg cost from `token_estimate` (typical of agentic chat sessions where the agent issues many short messages and emits long structured replies). Source: published vendor rates as of May 2026; captured at run time per model in `runs/<model>/_rates.json`.

Model	Input $/M	Output $/M	Mix 30/70 $/M	Source
Claude Opus 4.8	5.00	25.00	19.00	Anthropic [3]
GPT-5.5	5.00	30.00	22.50	OpenAI [4]

Claude Opus 4.8 is the *cheaper* model per token at the 30/70 mix ($19/M vs GPT-5.5's $22.5/M, a 0.84× ratio). Despite that rate advantage, most Opus 4.8 cells cost *more* end-to-end than their GPT-5.5 counterparts because the Opus 4.8 legs emit more tokens (cf. Table S5 in the main text): on the rate-card path Cursor ($13.23 vs $9.51 three-bench), OpenSpec ($13.05 vs $9.84), and Ralph ($13.84 vs $8.75) are all pricier under Opus 4.8, the per-token discount outweighed by volume. The widest gaps are on the heavy-token tools: Ralph's video-support cell ($5.66 Opus 4.8 vs $3.42 GPT-5.5, a 0.60× ratio) because the Opus 4.8 loop emits roughly double the tokens of its GPT-5.5 counterpart. On the cache-aware (provider-reported) OpenCode cells the shift is the most extreme in the study: the blog bench Opus 4.8 cell bills $49.57 against GPT-5.5's $8.38 (0.169×), entirely because the Opus 4.8 OpenCode greenfield run is a 40.1M-token deep-context session; the video-support bench OpenCode cell is similar ($22.07 vs $3.85, 0.175×). OpenCode is therefore the most token-intensive tool under Opus 4.8 across these benches, not the bargain it is under GPT-5.5, though the extreme blog-bench figure ($49.57) is a single unrepeated observation pending replication.

7.2. Per-parameter Δ matrix

Table S8 reports the per-parameter score deltas (GPT-5.5 minus Claude Opus 4.8) for every cell.

Reading the Δ matrix.

Five parameters carry every non-zero entry. Param 8 (Latency) moves only on the `openspec` blog cell: +2 (Opus 4.8's 1166 s anchor-3 vs GPT-5.5's 212 s anchor-5). The Opus 4.8 `openspec` single-file and video-support cells record longer raw wall spans (2335 s and 23663 s), but those are attended-session measurement artefacts (Section 4.3) scored anchor-5 under both models, so they carry no latency delta. No other tool moves on latency: `cursor`, `ralph`, and OpenCode all stay inside the anchor-5 band (< 10 min) under both models on all three benchmarks. Param 9 (Token cost) lifts toward GPT-5.5 on the token-heavy cells: `cursor` +1 (single-file, blog), `openspec` +1 (single-file), `ralph` +1 (blog), and OpenCode +2 on the single-file and video-support benches and +1 on the blog bench (the Opus 4.8 OpenCode cells are the cost outliers of the study). Param 5 (Code-reference accuracy) is mixed-sign and is the parameter most sensitive to plan precision: GPT-5.5 scores +1 on the OpenCode single-file bench (where the Opus 4.8 plan carries the 47-vs-53 `team_access` count error) but −1 on Cursor single-file, OpenSpec video-support, Ralph video-support, and Cursor blog, Ralph blog, and OpenCode blog (where the Opus 4.8 plan body edges GPT-5.5's). The blog bench adds the only Param 2 and Param 4 movers in the study: Ralph's blog cell loses 2 on Param 2 (Requirement traceability) and 1 on Param 4 (Test-plan coverage) under GPT-5.5 because the C1–C26 capability matrix and its per-task test linkage (present under Opus 4.8) collapse to a free-text bullet list under GPT-5.5; OpenSpec and OpenCode each move ±1 on Param 4 on the blog bench. These blog-bench Param 2/4 swings are what make Ralph's blog Δ the

Table S8. Per-parameter score deltas, GPT-5.5 minus Claude Opus 4.8, on the sixteen-parameter rubric. A positive value means GPT-5.5 out-scored Opus 4.8 on that parameter. "–" marks an integer-equivalent cell (the two models land on the same anchor); small non-zero Δ Totals on otherwise-dashed rows reflect sub-anchor weighting rounding. The non-zero entries fall on five parameters: 2 (Requirement traceability, the Ralph blog capability matrix), 4 (Test-plan coverage, on the blog bench), 5 (Code-reference accuracy), 8 (Latency), and 9 (Token cost). Crucially, Param 13 (Human-in-the-loop validation) carries *no* cross-model delta on any cell, because OpenSpec and Ralph run Cline-as-extension under *both* models (driver held constant); columns 10–12 and 14–16 are "–" across the board because they score the tool surface, which is unchanged across models. Δ Total values are the canonical per-cell weighted differences from `tools/dump_dual_primary.py` over `scoring/scores-opus48.csv` and `scoring/scores-gpt55.csv`.

Tool	Bench	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	Δ Total
`cursor`	single-file	–	–	–	–	−1	–	–	–	+1	–	–	–	–	–	–	–	−0.040
`cursor`	video-support	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	−0.010
`cursor`	blog	–	–	–	–	−1	–	–	–	+1	–	–	–	–	–	–	–	−0.035
`openspec`	single-file	–	–	–	–	–	–	–	–	+1	–	–	–	–	–	–	–	+0.030
`openspec`	video-support	–	–	–	–	−1	–	–	–	–	–	–	–	–	–	–	–	−0.080
`openspec`	blog	–	–	–	−1	–	–	–	+2	–	–	–	–	–	–	–	–	−0.035
`ralph`	single-file	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	–	−0.010
`ralph`	video-support	–	–	–	–	−1	–	–	–	–	–	–	–	–	–	–	–	−0.080
`ralph`	blog	–	−2	–	−1	−1	–	–	–	+1	–	–	–	–	–	–	–	−0.350
`opencode`	single-file	–	–	–	–	+1	–	–	–	+2	–	–	–	–	–	–	–	+0.135
`opencode`	video-support	–	–	–	–	–	–	–	–	+2	–	–	–	–	–	–	–	+0.060
`opencode`	blog	–	–	–	+1	−1	–	–	–	+1	–	–	–	–	–	–	–	+0.070

largest in the study (−0.350). Critically, Param 13 (Human-in-the-loop validation) carries *no* delta on any cell, including the blog cells, where OpenSpec and Ralph are scored 4/4 under both models on the same Cline-refinement basis as their single-file/video-support cells, because OpenSpec and Ralph run Cline-as-extension under *both* models and the HiL surface is identical across the model axis. The remaining parameters (1, 3, 6, 7, 10, and the tool-surface parameters 11, 12, 14, 15, 16) are model-stable on every cell.

Implication for tool selection.

The rubric's headline properties are near-rank-stability (no per-tool ranking flips on the single-file and video-support benches, with the *single* flip in the study on the blog bench where Ralph and OpenSpec swap the top two) and the cost-and-latency shift under model substitution. Teams who plan to rotate through frontier models over a project's lifetime should treat the per-tool ranking on the artefact-and-process and tool-surface dimensions as essentially model-invariant on existing-codebase tasks, but should expect breadth-oriented greenfield tasks to be more model-sensitive: the cross-model deltas are confined to Latency (an OpenSpec-specific Opus 4.8 outlier), Cost (token-volume driven), a ±1 Code-reference swing, and the blog-bench Param 2/4 swing that drives the one rank flip. Teams targeting the cheapest end-to-end run should prefer OpenCode on GPT-5.5 on the video-support bench ($3.85 cache-aware, the lowest in the matrix) and be cautious with OpenCode under Opus 4.8 on token-heavy work ($49.57 on the blog bench, the highest in the study, though a single-run figure pending replication); Cursor is the cheapest rate-card tool under both models on the single-file bench ($2.71 Opus 4.8, $2.33 GPT-5.5, on top of the Cursor Pro floor).

7.3. Operator-experience stability

The operator-experience companion rubric in Table 3 is model-stable for every tool, because each tool's *driver* is held constant across the model axis:

- For `cursor`, the IDE chat surface is the same under both models; the only difference is the model picker. The OPX score holds at 4.75 under both.
- For `openspec` and `ralph`, both model conditions run Cline as a Cursor / VS Code extension (operator-attended), so the setup friction, time-to-first-viable-plan, in-flight iterability, and cognitive-load parameters (O1–O4) are identical across the model axis. The underlying file conventions are the tool's (4-artefact change directory for OpenSpec; `RALPH_TASK.md` loop semantics for Ralph), and they do not change with the model.
- For OpenCode, both model conditions run the tool's own native headless runtime (`opencode run -m <provider/model>`), so its OPX is likewise model-stable.

Because the driver is constant per tool, the two OPX columns in Table 3 are equal and the cross-model average equals each column; the rubric isolates a pure (tool) effect rather than a (tool, driver) interaction.

7.4. Headline cross-model reading

1. **The rubric is robust to model substitution on eleven of twelve cells.** Per-cell deltas range from -0.350 to $+0.135$ and are mixed-sign; no per-tool ranking flips on the single-file or video-support benches, and the only flip in the study is on the blog bench (Ralph $\leftrightarrow$ OpenSpec). There is no Param 13 (HiL) contribution because the driver is held constant; the shifts are concentrated in Param 8 (Latency, an OpenSpec-specific Opus 4.8 outlier) and Param 9 (Cost), with a ± 1 Param 5 (Code-reference) swing on most cells and a larger Param 2/4 swing on the Ralph blog cell (its Opus-only capability matrix).
2. **The video-support bench cross-model average favours OpenCode.** Complex-bench cross-model totals are OpenCode 3.955, `ralph` 3.855, `openspec` 3.830, `cursor` 3.430. OpenCode's lead comes from anchor-5 on Params 3, 4, 5 (Task granularity, Test-plan coverage, Code-reference accuracy) under both models plus Param 6 (MCP integration) at anchor-5 (17 verified Atlassian + GitHub calls under Opus 4.8, 16 under GPT-5.5). Cursor's video-support bench Param 6 also lands at anchor-5 (12 verified calls under Opus 4.8, 16 under GPT-5.5), which is the single clearest signal that the canonical-pairing tools and the operator-attended chat surface exhaust the same external grounding when given the same ticket. Among the Cline-driven tools the tool-surface parameters (notably Param 14, Autonomous capacity, where only `ralph` anchors at 5) are what keep Ralph ahead of OpenSpec on the video-support bench.
3. **Defect avoidance is mostly spec-format-bound, with one model-dependent exception.** The regex passthrough trap fires (or doesn't) chiefly by tool, not by model (Table 2 in the main text): it lands on Cursor and Ralph under both models and never on OpenSpec. The exception is OpenCode, which avoids the trap under Opus 4.8 (its plan diagnoses the digit and broadens the regex up front) but falls into it under GPT-5.5, the only cell where frontier-model substitution changes the trap outcome. OpenSpec's spec format avoids it under both models without inspecting the digit.
4. **Opus 4.8 is cheaper per token yet pricier end-to-end on most cells.** At the 30/70 mix Opus 4.8 ($19/M) undercuts GPT-5.5 ($22.5/M), but the Opus 4.8 legs emit more tokens, so Cursor, OpenSpec, and Ralph all cost more end-to-end under Opus 4.8 on the rate-card path. The OpenCode cells are the extreme case: the provider-reported Opus 4.8 blog cell bills $49.57 against GPT-5.5's $8.38 because the Opus 4.8 OpenCode run is a 40.1M-token deep-context session. Teams choosing tools by cost should not adopt a single "cheapest tool" narrative without specifying the model: under GPT-5.5 OpenCode is the cheapest tool on the video-support bench ($3.85 cache-

aware), while under Opus 4.8 OpenCode is the most expensive on every bench and Cursor is the cheapest rate-card tool on the single-file bench.

5. **The two models differ on scope behaviour; the raw OpenSpec wall-time gap is a measurement artefact.** The Opus 4.8 OpenSpec planning legs record much longer raw wall spans (23663 s video-support, 2335 s single-file) than the same cells under GPT-5.5 (269 / 181 s), but those Opus spans are attended-session idle artefacts (Section 4.3) scored anchor-5 rather than genuine latency. The substantive model difference is scope behaviour: on the single-file bench the GPT-5.5 OpenSpec / Ralph / OpenCode loops autonomously remediate ambient `terraform fmt` drift where the Opus 4.8 cells deliberately scoped it out (Appendix 4.4). Teams adopting either model should expect to revise their scope expectations accordingly.

8. OpenCode cells

This appendix documents the six OpenCode cells in the matrix (one per benchmark × two models, across all three benchmarks). It records their motivation, the apparatus that drives them, the six-cell results, and the three cross-tool questions they answer.

8.1. Why a fourth tool

The threats-to-validity disclosure in Section 4.3 (*"OpenSpec and Ralph were measured under Cline, not their canonical drivers"*) names a runtime confound: the `openspec` and `ralph` cells in the matrix are run with Cline as the agent runtime, not the canonical drivers (Cursor / Claude Code / OpenCode for OpenSpec; an external agent runtime per iteration for Ralph). The token-cost cells, the operator-experience scores, and the MCP-call counts therefore partly capture Cline's framing rather than the upstream tool's own surface.

The OpenCode cells are the explicit counter-evidence. They use OpenCode's own native headless runtime (`opencode run`) with the same prompt fingerprints, the same MCP servers, and the same artefact schema as the rest of the study, so the reader can compare a canonical-pairing data point against the Cline-pairing data points elsewhere in the table. Concretely:

- OpenCode is a single binary that ships its own LLM client, MCP picker, session store, and headless invocation mode (invoked as `opencode run --model <provider/model> --format json`).
- No IDE extension, no runtime wrapper. The agent talks directly to the configured provider with the operator-supplied API key.
- MCP servers are wired in a per-bench `opencode.json` (GitHub MCP for the single-file bench; GitHub plus Atlassian for the video-support bench; GitHub MCP for the blog bench, where it is also the incremental-commit channel), so the picker behaviour is comparable to Cline's but the framing overhead is absent.

8.2. Cells and apparatus

Table S9 lists the six OpenCode cells and the helper script that drives each.

Table S9. The six OpenCode cells and the helper script that drives each, with observed wall times.

Cell	Bench	Model	Helper script	Wall (plan + exec)
7	github-bootstrap	Claude Opus 4.8	`run_cell7.sh opus`	98 s + 235 s = 333 s
7	github-bootstrap	GPT-5.5	`run_cell7.sh gpt`	119 s + 184 s = 303 s
8	worker-video-feature	Claude Opus 4.8	`run_cell8.sh opus`	293 s + 350 s = 643 s
8	worker-video-feature	GPT-5.5	`run_cell8.sh gpt`	188 s + 253 s = 441 s
12	personal-blog-scaffold	Claude Opus 4.8	`run_cell12.sh opus`	118 s + 1724 s = 1842 s
12	personal-blog-scaffold	GPT-5.5	`run_cell12.sh gpt`	123 s + 1245 s = 1368 s

The helper scripts implement the same six-step life-cycle as `tools/run_cell{1..6}.sh`: prep, plan-run, capture-plan, exec-run, capture-exec, drop. Two things differ. First, the `plan-run` and `exec-run` sub-commands invoke `opencode run` directly rather than handing off to an IDE extension; the operator is unattended for the full duration of each leg. Second, the artefact set adds two OpenCode-specific files per leg: `transcripts/opencode-stdout.log` (the streaming JSON event log from `opencode run` with the JSON format flag) and `transcripts/session-export.json` (the deterministic export fetched via `opencode export` on the captured session id). Token totals and per-tool-call cost figures come from `session-export.json` rather than from a heuristic byte-volume estimate, removing the Parameter 9 caveat that applies to the Cline-driven cells. The full operator runbook (provider auth, MCP auth, model-string verification, drop / redact / guard recipe) is at `HANDOFF-OPENCODE-RUN.md` in the study root. The MCP picker configurations are at `tools/opencode/opencode.simple.json` and `opencode.complex.json`; the headless wrapper is at `tools/opencode/opencode_run.sh`.

8.3. Per-cell weighted totals and per-parameter scores

Table S10 gives the per-cell weighted totals and Table S11 the full per-parameter scores for the six OpenCode cells.

Table S10. Per-cell weighted totals (max 5.0) for the six OpenCode cells. Across all three benchmarks OpenCode ranks third on the cross-model average (3.701, vs OpenSpec's 3.798 and Ralph's 3.728 in Table 1): the blog bench has no Jira ticket and no rich external MCP for OpenCode to use (Param 2 and Param 6 sit at the no-external-source baseline), so its blog cell is third under both models (3.540 cross-model). OpenCode wins the video-support bench outright (3.955 cross-model vs Ralph's 3.855 and OpenSpec's 3.830).

Bench	Opus 4.8	GPT-5.5	Cross-model avg
`github-repo-bootstrap`	3.540	3.675	3.608
`worker-video-feature`	3.925	3.985	**3.955**
`personal-blog-scaffold`	3.505	3.575	3.540
three-benchmark average	3.657	3.745	3.701

8.4. Three cross-tool questions, answered

Q1. Does the Cline confound move the rubric ranking?

No: where the canonical and Cline pairings are directly comparable (the externally-grounded video-support bench) the native pairing *out-scores* the Cline-driven tools; the cross-tool average leader changes only because of benchmark composition, not the driver. Compare OpenCode's per-cell weighted total against the `openspec` and `ralph` totals in Table 1: on the video-support bench OpenCode leads under both models (Opus 4.8 3.925 vs `openspec` 3.870 vs `ralph` 3.895; GPT-5.5 3.985 vs 3.790 vs 3.815). On the single-file bench under Opus 4.8 OpenCode (3.540) sits *below* `openspec` (3.695); on the blog bench OpenCode is third under both models (3.505/3.575) because the empty repository strips the external grounding it depends on. The cross-model three-benchmark average therefore favours OpenSpec (3.798) over Ralph (3.728) and OpenCode (3.701), but this is a benchmark-composition effect: on the one bench where integration demand makes the canonical vs Cline pairing directly comparable, OpenCode's native pairing is on top, so the Cline-pairing confound is not *inflating* OpenSpec's or Ralph's scores past OpenCode's. The Cline-pairing caveats (Cline's prompt-assembly overhead inflates Param 9 cost estimates; Cline's chat surface partly drives the OPX scores; Cline's MCP auto-injection partly drives the Param 6 MCP-call counts) remain literally true, but the cross-tool average

Table S11. Per-parameter scores for the six OpenCode cells (single-file, video-support, and blog benches under both models). Column abbreviations: "s" = single-file, "c" = video-support, "b" = blog; "/O" = Opus 4.8, "/G" = GPT-5.5. Anchor-5 highlights: OpenCode anchors at 5 on Task granularity (3) and Hand-off friction (10) across all three benches and both models, on Test-plan coverage (4) on all but the Opus blog cell, and on Code-reference accuracy (5) and MCP integration (6) on the video-support bench; the blog bench drops Param 2 (no external ticket) and Param 6 (GitHub-MCP-only) to the no-external-source baseline, which is why the blog columns sit below the video-support columns.

#	Parameter	s/O	s/G	c/O	c/G	b/O	b/G
1	Artifact completeness	2	2	2	2	2	2
2	Requirement traceability	2	2	4	4	2	2
3	Task granularity	5	5	5	5	5	5
4	Test-plan coverage	5	5	5	5	4	5
5	Code-reference accuracy	4	5	5	5	5	4
6	MCP / external-source integration	3	3	5	5	3	3
7	Reproducibility (artefact persistence)	4	4	4	4	4	4
8	Latency to ready-to-implement	5	5	5	5	5	5
9	Token / API cost	2	4	1	3	2	3
10	Hand-off friction to implementation	5	5	5	5	5	5
11	Tool licensing & accessibility	5	5	5	5	5	5
12	Learning curve / time-to-productivity	3	3	3	3	3	3
13	Human-in-the-loop validation experience	2	2	2	2	2	2
14	Autonomous capacity	2	2	2	2	2	2
15	Runtime integration & ergonomics	3	3	3	3	3	3
16	Code-review / hand-off review surface	4	4	4	4	4	4
	Weighted total	**3.540**	**3.675**	**3.925**	**3.985**	**3.505**	**3.575**

leader is determined by the benchmark composition (in particular the breadth-oriented blog bench), not by the driver.

Q2. Does the canonical pairing change Parameter 9 (Token / API cost)?

Yes, and the change is consistent with the "rate-card overestimates real spend on heavy-cache workloads" hypothesis. The OpenCode session export under Opus reports billed tokens deterministically, and the GPT-5.5 cells are reconstructed by the per-line-rate backfill in `opencode_run.sh` so each token line item (input, output, cached read, cached write) is priced separately. OpenCode's cache-aware totals are materially *lower* than the rate-card estimates would predict despite OpenCode emitting the most tokens of any tool: video-support bench Opus 4.8 is $22.07 cache-aware vs rate-card-equivalent $286.94, and the blog bench Opus 4.8 is $49.57 vs a rate-card-equivalent ∼$763 (its 40.1 M tokens at $19/M). The delta is the framing overhead *plus* the prompt-caching discount that the rate-card path does not model. A second reading specific to the Opus 4.8 column: although the cache-aware figures are far below rate-card, their *absolute* level is the highest in the study: the blog cell at $49.57 (vs $8.38 for the same cell under GPT-5.5) is the single most expensive cell in the corpus, because the Opus 4.8 OpenCode greenfield run is a 40.1 M-token deep-context session. OpenCode is thus the most token-intensive tool under Opus 4.8 across these benches, though the extreme blog-bench figure ($49.57) rests on a single un-

repeated run and should be treated as a candidate anomaly pending replication, and the cheapest tool on the video-support bench under GPT-5.5; the cost ordering across tools cannot be read without specifying the model.

Q3. Does OpenCode reproduce the regex-passthrough trap on the video-support bench?

Only under GPT-5.5; under Opus 4.8 OpenCode *avoids* it. The trap (failure to extend `STANDARD_PATH_REGEX` from `[a-z]+` to `[a-z0-9]+` so that `mp4|webm|mov` would route correctly) is the one cell in the matrix where frontier-model substitution changes the outcome. Under **Opus 4.8** the OpenCode planning leg diagnoses the defect directly: the plan body states that "`mp4` contains a digit and fails the regex … the group must become `([a-z0-9]+)`" (`OPENCODE_PLAN.md` Section 2.3) and Task 2 broadens the character class before any code is written, so the trap never fires. Under **GPT-5.5** the same tool asserts the regex was already permissive (plan body: `[a-z]+` "already allows mp4/webm/mov"), falls in, and the execution leg recovers via a Vitest failure during apply by widening the character class one character. Recovery imposed no test regressions. The defect-convergence story in Section 3.2 therefore reads: *the spec format avoids the trap under both models without inspecting the digit (OpenSpec), while plan-content quality can avoid it only when it is high enough, and is model-dependent*. OpenCode's Opus 4.8 plan is the single most precise code-reference diagnosis in the study and avoids the trap, but the same tool falls in under GPT-5.5; only the 4-artefact spec scaffold front-loads "parser broadening" as a first-class requirement under both models.

8.5. Cross-model behavioural asymmetry on OpenCode

A cross-model behavioural difference surfaces on the single-file bench OpenCode cells: *precondition-mismatch resolution scope*. The single-file bench prompt asks the planner to verify `terraform fmt -check -recursive` runs clean, and the target repo carries 13 pre-existing unformatted files unrelated to the requested change. OpenCode/Opus deliberately scoped its fix to the new file (recording the global drift as a pre-existing condition in `deviations.json`, exit-0 verifying only the target subdirectory); OpenCode/GPT-5.5 ran a repo-wide `terraform fmt -recursive` to clear all 13 files of drift before re-running the strict-clean postcondition. Both responses produce a green strict-clean postcondition; neither violates the test plan. The same scope-resolution split surfaces on the OpenSpec and Ralph single-file bench cells under GPT-5.5 (and on neither of their Opus equivalents), confirming this is a model-bound asymmetry that generalises across all canonical-pairing tools rather than a tool-specific OpenCode behaviour: GPT-5.5 pulls scope outward toward strict-postcondition closure, Opus pulls scope inward toward single-file-change minimality. Recorded in Appendix 4.4 with the corresponding diff-stat asymmetry in Table S3.

9. Repeat-run variance for the close-margin cells

The main-article ranking turns, under Opus 4.8, on a 0.010-point margin between `openspec` (3.812) and `ralph` (3.802) in the three-bench average (Section 3). To estimate the run-to-run variability of that margin we repeated the cells that determine it, `openspec` and `ralph` under Claude Opus 4.8 on all three benchmarks (planning and execution legs), *two additional times each*, for 12 additional attended captures under a frozen protocol (identical prompt bodies, model pins, and rubric). The original captures are retained unchanged as *repeat-0*, so the aggregate table (Table 1) is untouched; the repeats are scored on the same frozen sixteen-parameter rubric (Appendix 1).

Scoring method.

A parameter is allowed to move across repeats only when the repeat artefact genuinely differs from repeat-0, never through scorer-calibration drift. The mechanical parameters (Parameter 8 latency, Parameter 9 cost) are recomputed from each repeat's measured planning wall time (`started_at` → `ended_at`) and token total; the tool-surface parameters (11–16) are driver-determined and held constant, since the Cline-as-extension driver is unchanged; and the plan-content parameters (1–7, 10) are re-read from each repeat's planning artefacts against the frozen anchors. Re-summing the frozen repeat-0 vectors reproduces every published aggregate total exactly. Only two genuine plan-content moves were found: the `openspec` blog repeat-1 dropped Parameter 5 to anchor-4→3 on a one-off cross-artefact naming drift (a `blog` content collection in one spec while the rest of the plan used `/articles/`), and the `ralph` video repeat-1 dropped Parameter 6 to anchor-5→4 because a ticket-linked reference file returned HTTP 404 during that run; both were coherent again in repeat-2. All other movement is mechanical latency/cost.

Latency artefact check.

The repeats also bear on the two attended-session latency artefacts noted in Section 3. The frozen repeat-0 `openspec` planning legs on the video-support and single-file benches recorded raw spans of 23,663 s (≈ 394 min, left open overnight) and 2,335 s (≈ 38.9 min, start stamp predating the Send), which we scored at anchor-5 as measurement artefacts. The four clean re-runs confirm that treatment: the same planning legs completed in 6.0 and 9.8 min (video-support) and 9.2 and 3.5 min (single-file), all inside the anchor-5 (<10 min) band, so the original spans reflect attended-session idle time rather than active planning.

Table S12. Per-cell weighted engineering-rigor total (max 5.0) across the three replicates under Claude Opus 4.8, for the six close-margin cells. *r0* is the frozen published cell (Table 1); *r1* and *r2* are the two additional repeats. The largest per-cell spread is 0.105 (`ralph`/video, driven by a planning-latency swing of 7.5→12.6→16.9 min moving Parameter 8 across anchor 5→4→3); `openspec`/video is identical across all three replicates.

Tool	Benchmark	r0	r1	r2	mean	range
`openspec`	single-file	3.765	3.800	3.800	3.788	0.035
`openspec`	video-support	3.870	3.870	3.870	3.870	0.000
`openspec`	blog	3.800	3.765	3.835	3.800	0.070
`ralph`	single-file	3.475	3.510	3.510	3.498	0.035
`ralph`	video-support	3.895	3.790	3.825	3.837	0.105
`ralph`	blog	4.035	4.035	4.070	4.047	0.035

Table S13. Three-bench average (avg-3) per tool per replicate index under Opus 4.8, and the OpenSpec−Ralph margin. `openspec` leads `ralph` in all three replicates, but the margin (0.010–0.033) is of the same magnitude as the per-cell run-to-run range in Table S12 (up to 0.105), so under Opus 4.8 the two tools are within run-to-run variation and no ordering is claimed between them.

Replicate	`openspec` avg-3	`ralph` avg-3	margin (os−ralph)	leader
r0 (frozen)	3.812	3.802	+0.010	`openspec`
r1	3.812	3.778	+0.033	`openspec`
r2	3.835	3.802	+0.033	`openspec`
pooled (min–max)	3.812–3.835	3.778–3.802	—	—

Each tool's avg-3 varies by 0.023 across the three replicates. The cross-model three-bench average (OpenSpec ahead of Ralph), which does not rest on the Opus-only margin, is unaffected. All *other* cells in the 48-cell matrix remain single-run point

estimates (Section 4.3). The full per-parameter repeat vectors, with per-row provenance notes (mechanical / tool-surface / plan-content / content-move), are released as `scoring/scores-opus48-repeats.csv`; the compact totals and the analysis narrative are in `scoring/variance-repeats.csv` and `scoring/variance-analysis.md`. The 12 full repeat captures (planning and execution artefacts, timing, and token counts) are released under `metadata/runs/claude-opus-4-8/⟨benchmark⟩/⟨tool⟩/repeat-{1,2}/` in the public artefact repository alongside these scoring files.

References

1. Cline. Cline Pricing. https://cline.bot/pricing, 2026. Public pricing page consulted for the Tool licensing & accessibility parameter (Param 11). Open-source frontend; MCP transports available on the free tier; the operator pays only the underlying provider's per-token rate.
2. Anysphere. Cursor Pricing. https://cursor.com/pricing, 2026. Public pricing page consulted for the Tool licensing & accessibility parameter (Param 11). Hobby tier free; Pro $20/mo unlocks the MCP picker required by both Cursor cells in this study.
3. Anthropic. Claude Opus 4.8: System Card and Capabilities. https://www.anthropic.com/news/claude-opus-4-8, 2026. First primary model condition; used for sixteen of the 32 (tool × benchmark × leg) cells in this study. Standard-mode pricing: $5.00/M input + $25.00/M output (weighted $19.00/M at the 30/70 mix); $0.50/M cached read; $6.25/M cached write.
4. OpenAI. GPT-5.5: Frontier Model Card and Pricing. https://openai.com/index/introducing-gpt-5-5/, 2026. Second primary model condition; used for sixteen of the 32 (tool × benchmark × leg) cells in this study. Pricing: $5.00/M input + $30.00/M output; $0.50/M cached read; $6.25/M cached write.